value does not depend on `n`), such that `g(n) ≤ c * f(n)` for every possible value of `n` (possibly larger than a certain value n_0).

For instance, if `f(n) = n` and `g(n) = 40 + 3*n`, we can choose `c = 4` and n_0 `= 40`.

Figure B.2 shows how these three functions grow with respect to each other: while `f(n)` is always smaller than `g(n)`, `c*f(n)` becomes larger than `g(n)` at some point. To better understand the turning point, we can evaluate the two formulas by substituting actual values for their parameter `n`; we use the following notation `f(1) -> 1` to assert that `f(1)` evaluates to `1` (in other words, the result of calling `f(1)` is equal to `1`).

For `n` lower than `40`, `g(n)` will be larger: for `n = 30`, `f(30) -> 120` and `g(n) -> 130`. Instead, we know that `f(40) -> 160` and `g(40) -> 160`, `f(41) -> 164` while `g(41) -> 163`. For any value greater than `41`, `40 + 3 *n <= 4 * n`.